Persistent

Module 3 & 4: DML Commands, Constraints & Relationships

Prerequisites:

Module 1 – Database Fundamentals

Module 2 – DDL Commands, MySQL via Docker.

Learning Objectives

Module 3 – DML

- Insert data into tables
- Update existing records
- Delete records
- Retrieve data using SELECT
- Filter and sort data

Module 4 – Constraints & Relationships

- Database constraints
- Primary & Foreign Keys
- Table relationships

> "DDL creates the structure. DML works with the data."

Persistent

What is DML?

Data Manipulation Language (DML) manages the **data stored inside** database tables — not the structure itself. While DDL modifies the schema, DML modifies the rows within that schema.

INSERT

Add new records

UPDATE

Modify existing rows

DELETE

Remove records

SELECT

Retrieve data

Persistent

INSERT Command

Adds new records into a table. Follow column order and use correct data types. Always avoid duplicate Primary Keys.

Single Record

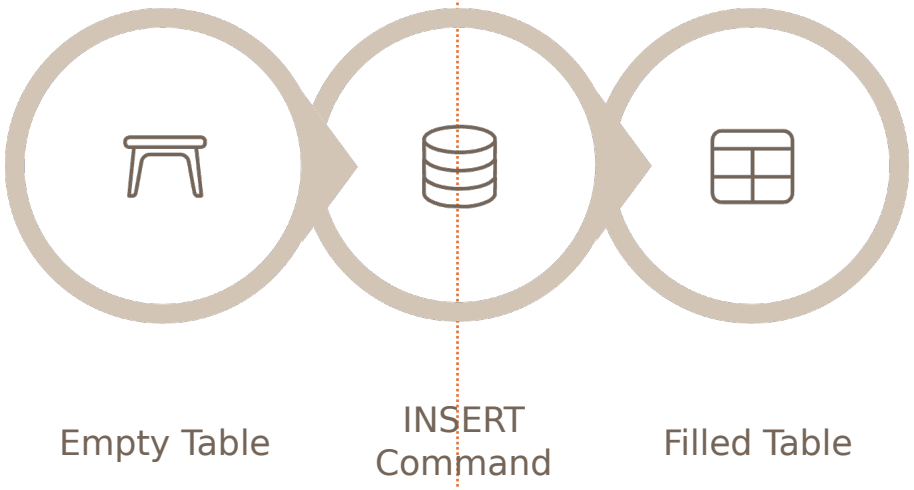
```
INSERT INTO employee VALUES
(101, 'John', 50000, 'HR');
```

Multiple Records

```
INSERT INTO employee VALUES
(102, 'David', 60000, 'IT'),
(103, 'Lisa',  55000, 'Finance');
```

Best Practices

- Insert correct data types
- Follow column order
- Avoid duplicate Primary Keys



Persistent

UPDATE & DELETE Commands

UPDATE – Modify Existing Records

```
UPDATE employee
SET salary = 65000
WHERE id = 102;
```

⚠ Omitting WHERE updates **every row** in the table. Always use WHERE unless intentional.

DELETE – Remove Records

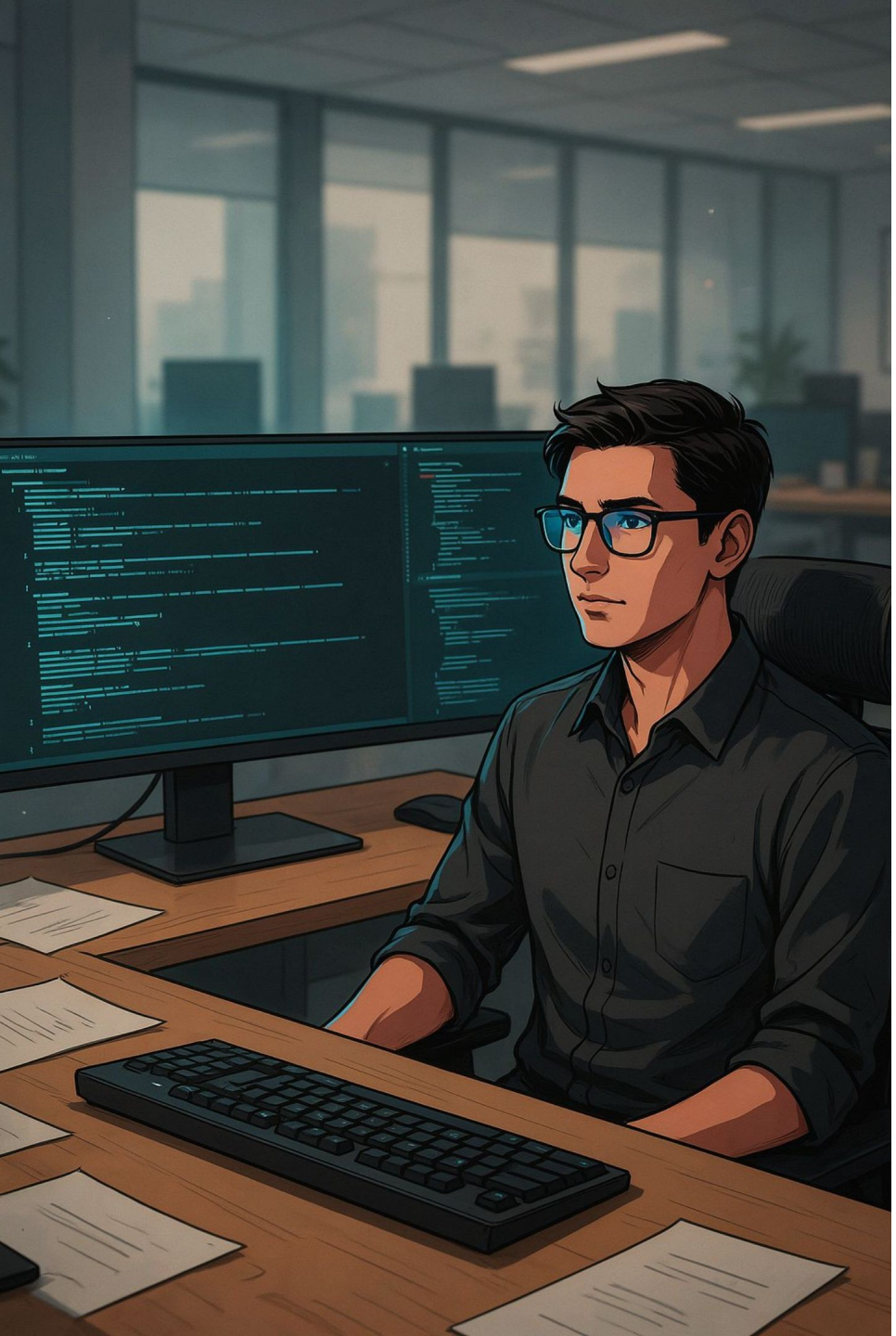
```
DELETE FROM employee
WHERE id = 103;
```

⊗ Without WHERE, **all rows** are deleted — but the table remains. DROP removes the table entirely.

DELETE vs DROP

DELETE removes rows; the table structure stays intact. DROP removes the entire table from the database.

Persistent



SELECT Command

Retrieve information from a table. Use `*` for all columns or specify column names for targeted output.

All Columns

```
SELECT * FROM employee;
```

Selected Columns

```
SELECT name, salary
FROM employee;
```

Query Flow

- Database receives query
- Engine scans the table
- Matching rows returned
- Result set displayed

Persistent

Filtering Data with WHERE

The WHERE clause retrieves only records that match a specified condition, reducing result sets to exactly what you need.

Example

```
SELECT * FROM employee
WHERE dept = 'IT';
```

Salary Filter

```
SELECT * FROM employee
WHERE salary > 50000;
```

Common Operators

= Equal	!= Not equal
> Greater	<= Less or equal

Persistent

Sorting, Limiting & Advanced Filtering

ORDER BY

```
ORDER BY salary ASC;
ORDER BY salary DESC;
```

LIMIT

```
SELECT * FROM employee LIMIT 5;
```

Advanced Filters

```
-- Remove duplicates
SELECT DISTINCT dept FROM employee;

-- Pattern matching
WHERE name LIKE 'A%';

-- Value list
WHERE dept IN ('HR','IT');

-- Range
WHERE salary BETWEEN 30000 AND 60000;

-- Null check
WHERE email IS NULL;
```

SQL Logical Operators

Combine multiple conditions using logical operators to build precise, powerful queries.

AND

Both conditions must be true.

```
WHERE dept='IT'
AND salary > 50000;
```

OR

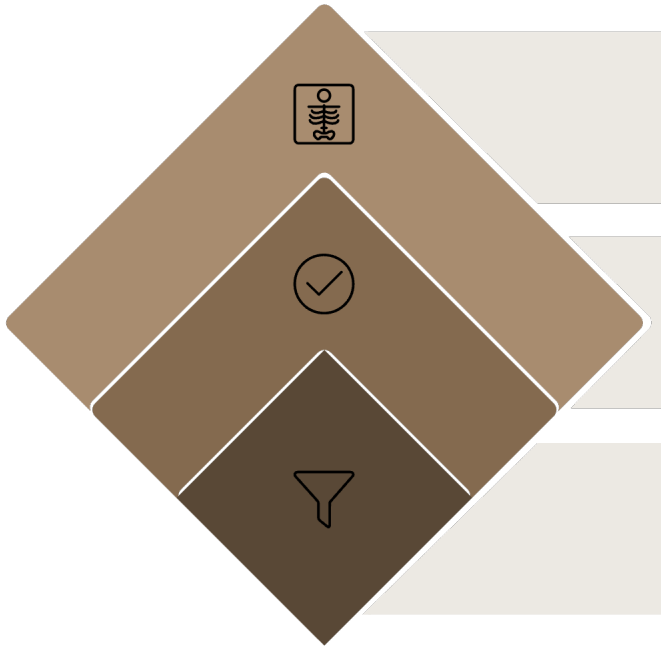
Either condition can be true.

```
WHERE dept='IT'
OR dept='HR';
```

NOT

Excludes matching records.

```
WHERE NOT dept='Finance';
```



False Branch

Skip the record

True Branch

Return matching record

Evaluate Condition

Check logical expression

Persistent

Introduction to Constraints

Constraints enforce rules on data stored in tables, maintaining integrity, preventing invalid entries, and ensuring consistency across the database.

Primary Key

Unique row identifier

Foreign Key

Links tables together

Unique

No duplicate values

Not Null

Value is mandatory

Default

Auto-fills a value

Check

Validates a condition

Auto Increment

Generates IDs automatically

Persistent

Primary Key & Foreign Key

Primary Key

- Must be **unique**
- Cannot be **NULL**
- Identifies exactly one row

```
Employee_ID  PRIMARY KEY
```

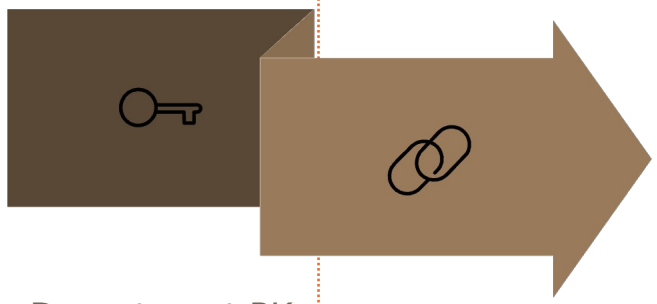
Foreign Key

Creates a relationship between two tables by referencing the Primary Key of a parent table.

```
Department_ID  FOREIGN KEY
  REFERENCES Department(Dept_ID)
```

- Employee Table → Child
- Department Table → Parent

Employee FK



Department PK

Persistent

Other Constraints in Practice

Beyond Primary and Foreign Keys, these constraints protect data quality at the column level.

1

UNIQUE

Prevents duplicate values. Example: `Email` column — no two employees share the same email.

2

NOT NULL

Makes a field mandatory. The record cannot be saved without a value in this column.

3

DEFAULT

Automatically inserts a preset value. Example: `Status = 'Active'` when no value is provided.

4

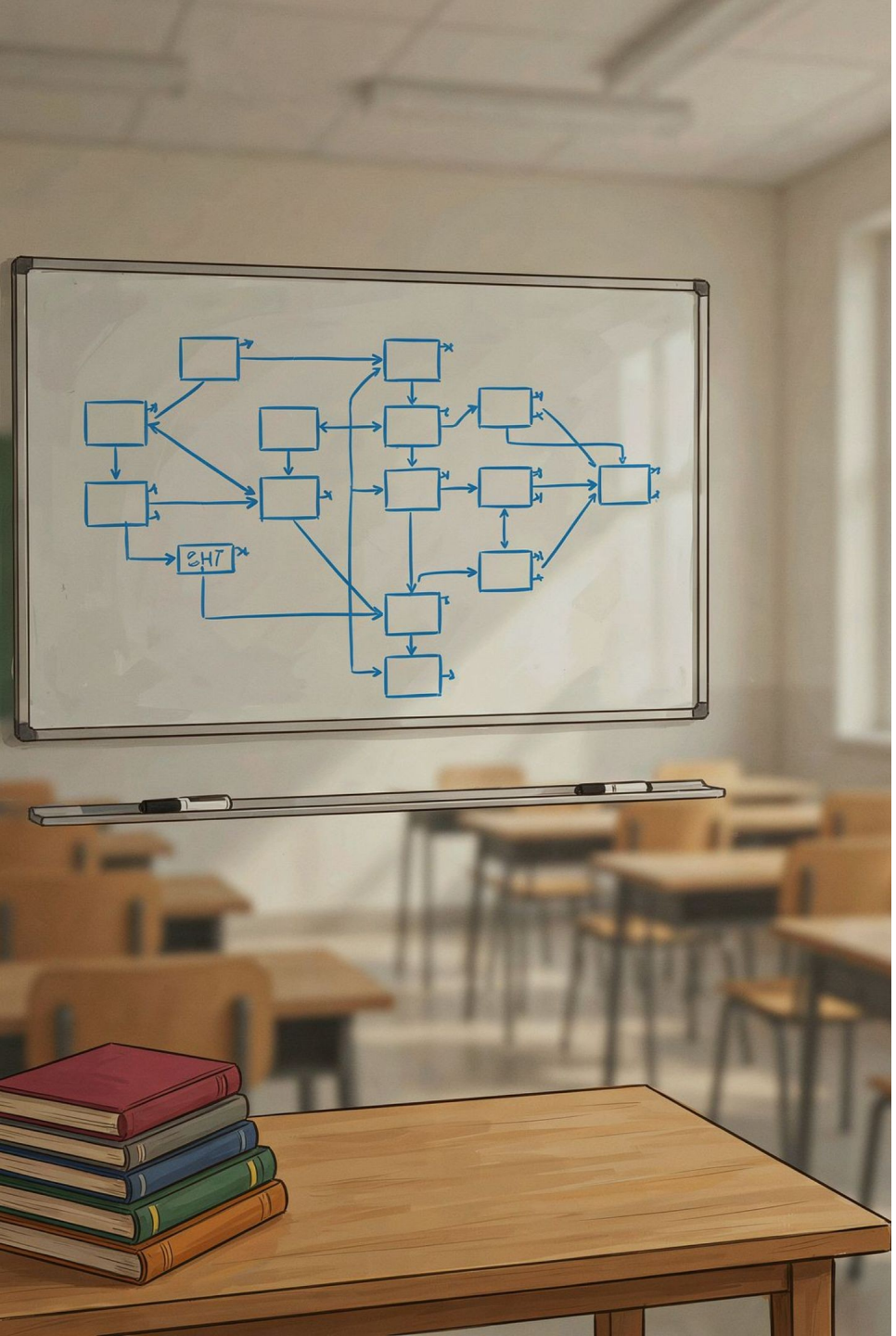
CHECK

Validates that values satisfy a condition. Example: `Salary > 0` — negative salaries are rejected.

5

AUTO_INCREMENT

Automatically generates sequential IDs for each new row inserted into the table.



Database Relationships

Relationships define how tables connect to each other. Choosing the right type ensures accurate data modeling.

One-to-One

One record in Table A links to exactly one record in Table B.

Example: One Passport → One Citizen

One-to-Many

One record in Table A links to many records in Table B.

Example: One Department → Many Employees

Many-to-Many

Multiple records on both sides. Requires a **Junction Table** to resolve the relationship.

Example: Students Courses

Persistent

Relationship Diagrams & Junction Tables

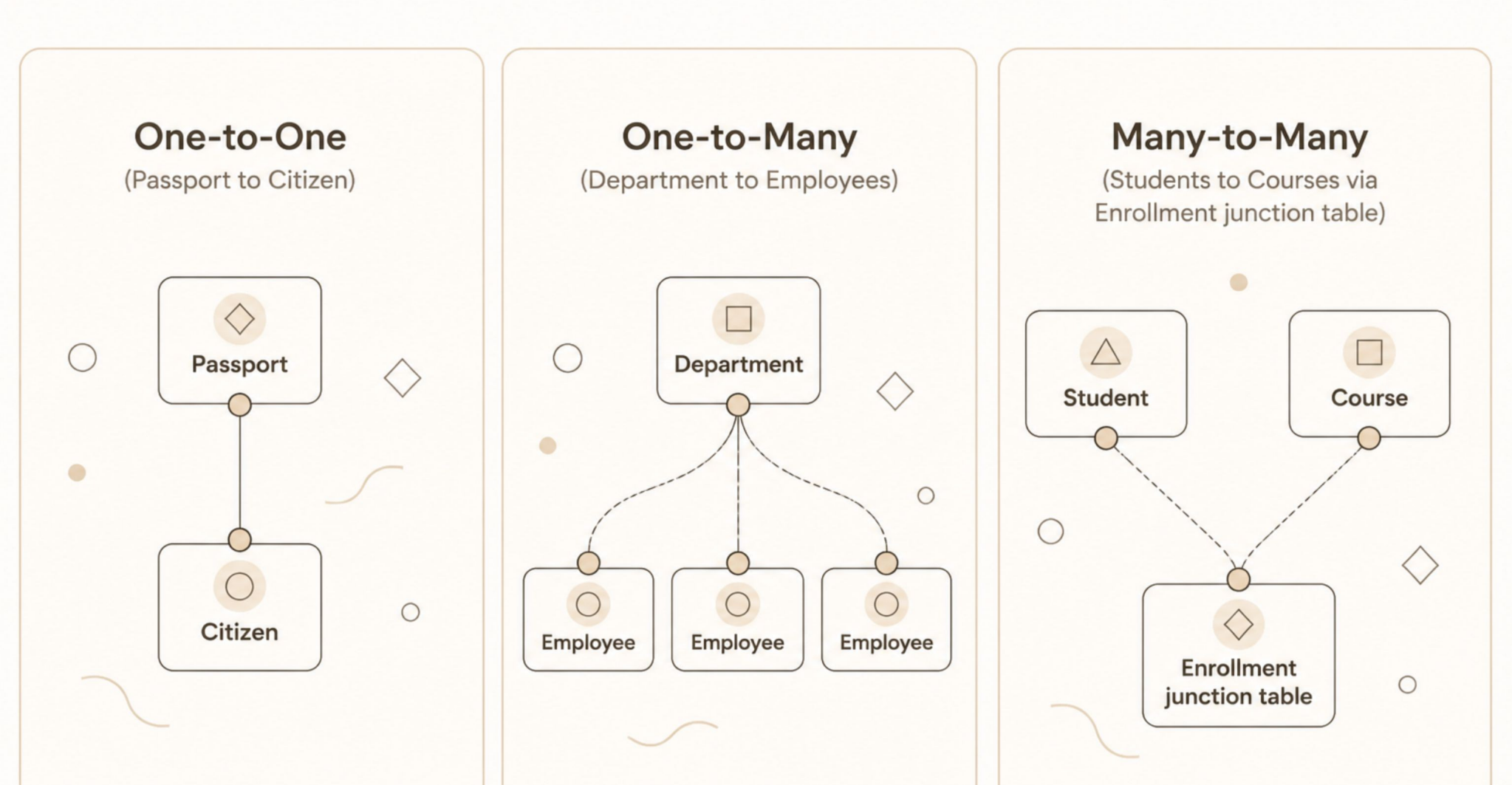
Many-to-Many: Students & Courses

A student can enroll in many courses, and a course can have many students. A junction table (`Enrollment`) resolves this.

```
Student(Student_ID, Name)
Course(Course_ID, Title)
Enrollment(Student_ID, Course_ID)
```

Why Junction Tables?

- Avoids data redundancy
- Stores relationship metadata (e.g., grade, date)
- Enforces referential integrity via Foreign Keys
- Enables complex JOIN queries



Persistent

Module Summary & Next Lab

> "A database is only as reliable as the rules that protect its data and the queries that retrieve it."

DML Commands Covered

- INSERT, UPDATE, DELETE, SELECT
- WHERE, ORDER BY, LIMIT
- DISTINCT, LIKE, IN, BETWEEN, NULL
- AND, OR, NOT operators

Constraints Covered

- Primary Key, Foreign Key, Unique
- Not Null, Default, Check, Auto Increment

Relationships Covered

- One-to-One, One-to-Many, Many-to-Many

Next Lab – Students Will:

- Insert employee records
- Update salaries
- Delete records
- Perform advanced SELECT queries
- Create parent-child tables
- Apply all constraints
- Build table relationships

Persistent

Persistent

Thank you